



SIEMENS
energy

DEF CON 34: ICS Hack 'n Track

Pedro Cabrera
Hannes Heck
Hannah Krebs
Sam Miorelli
Matt Parrott
Jordan Sanchez
Anthony Sevarino

Siemens Energy is a trademark licensed by Siemens AG.



Unrestricted © Siemens Energy, 2026 | Omnivise Cybersecurity



Let's make sure your laptop is ready to attack!

<https://github.com/hydrosam/DEFCON34-ICS-Workshop>



SIEMENS
energy

Distributed Control Systems 101

What is a Distributed Control System?

Plant-level control rather than just single-process controls

A DCS distributes control logic across many controllers close to the equipment they run, coordinated through a shared engineering and operator layer.

It has high redundancy including backup processors/servers and redundant networks.

The Human Machine Interface (HMI) is usually an integral part of a DCS (whereas with individual PLCs, the HMI could be entirely optional).

Unlike a SCADA system's centralized polling, each controller keeps running the process even if the network or the operator station goes down — boiler feedwater, turbine governing, and emissions loops don't stop just because someone in the control room lost a screen. DCS design optimizes for availability and determinism first; confidentiality is a distant third, the opposite priority order most IT security tooling assumes.

**Availability first.
Confidentiality
third.**

Why use a DCS instead of just individual PLCs or SCADA?



SIEMENS
ENERGY

Usually for complicated time-sensitive processes

Some processes are too complicated to control with the logic of a single PLC or a couple PLCs communicating just with each other.

DCS lets you have similar levels of high-sensitivity control that you get with a closed loop PLC but using many PLCs and other devices across big sites or complicated plants.

SCADA does not have the same time-sensitive messaging guarantees.

For plants like chemical plants, power plants, refineries, etc. often a DCS is the sweet spot for industrial controls.



SIEMENS
energy

Inside Omnivise T3000

What is Omnivise T3000?



SIEMENS
ENERGY

A DCS responsible for ~1/6 of all power generation worldwide.

Focused on energy infrastructure applications.

Integrated solution for:

- PLC's (mostly Siemens Simatic S7-400)
- Connections to other ICS's (i.e. for a water treatment system, fuel oil skid, and other auxiliaries)
- HMI's (called "Thin Clients" but actually full computers)
- Servers running windows to orchestrate and manage the devices
- Hygiene Cybersecurity solutions
- Networking equipment
- Remote access systems (the Terminal Server and stateful Scalance firewall)

Omnivise T3000 – Embedded Component Services™ powers the whole system



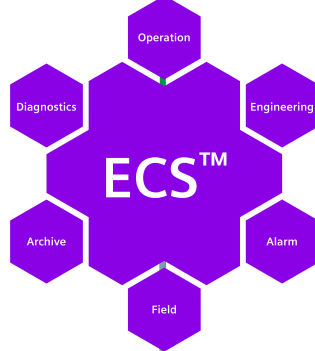
SIEMENS
ENERGY

T3000 architecture

User Interface



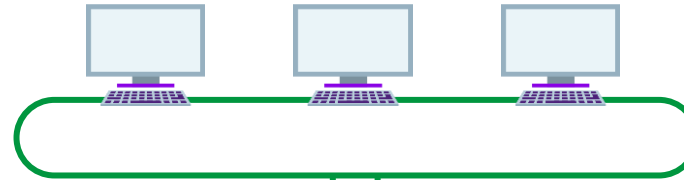
Server



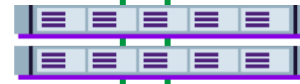
Process Interfaces

Hardware assignment

Thin Clients



Application Server



Automation Server



IO Devices

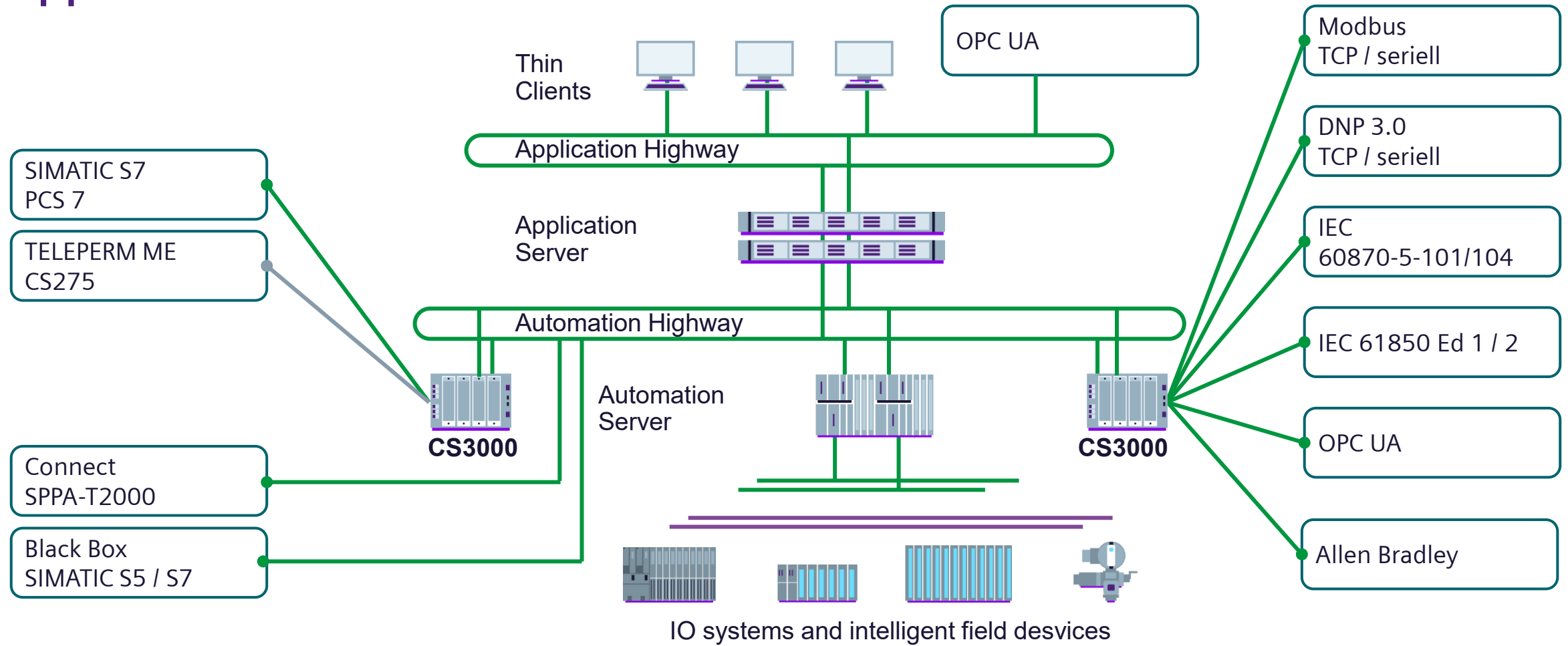
Minimal system complexity

- Flat system architecture with minimal amount of components and interfaces
- No sub-system boundaries, less administration efforts required
- Easy online upgrade and extension of components

Various and different interfaces to connect auxiliaries and 3rd party systems – a world of communication opportunities



SIEMENS
ENERGY



What Each Device Actually Does



SIEMENS
energy

- 1 Terminal Server
- 2 Application Server
- 3 PLC
- 4 T3000 Firewall (Scalance)

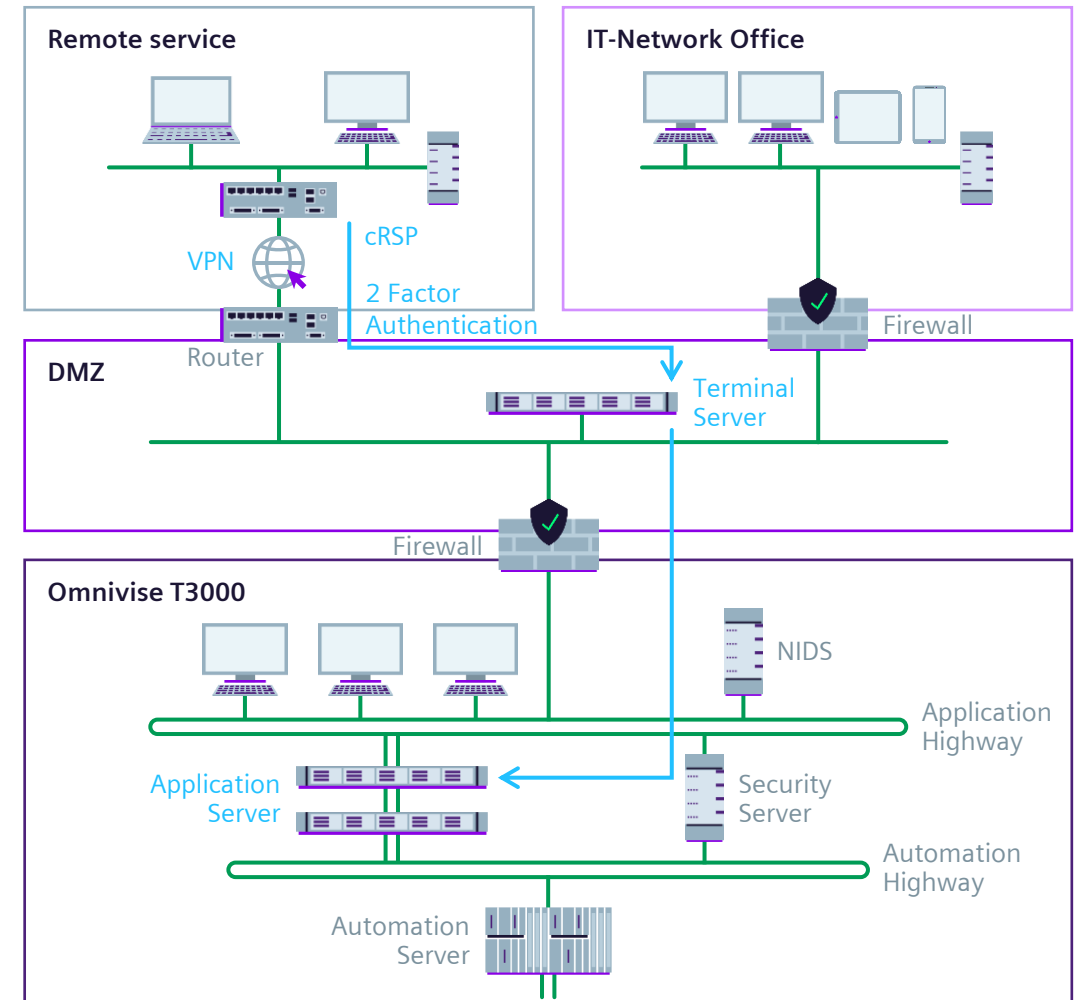


How T3000 Talks to the Outside World

Secure remote access when support is needed

Remote service is vital and must be secure

- Secure connection to Omnivise T3000 via Siemens Energy **cRSP** (common Remote Service Platform)
- **VPN** (virtual private network) tunnel protects communication against external access
- **2 Factor Authentication** requires strong authentication for remote access of the control system
- **Siemens Energy Remote Expert Center** can dial in and provide fast and reliable assistance 24/7/365 with guaranteed response times





From Fully Isolated to Fully Exposed

The T3000 firewall is meant to enforce this

Siemens Energy's documentation says external components should only reach the T3000 DMZ — never bridge directly to the Application or Automation highway.

That's a policy enforced by configuration, not a physical guarantee. If the Terminal Server picks up a second, internet-facing NIC during a rushed remote-support setup, a vendor shortcut, or weak segmentation discipline, **the isolation exists only on paper.**

- 1 Air-gapped
- 2 One-way historian push / data diode
- 3 Siemens cRSP
- 4 Two-firewall industrial DMZ
- 5 Direct internet exposure

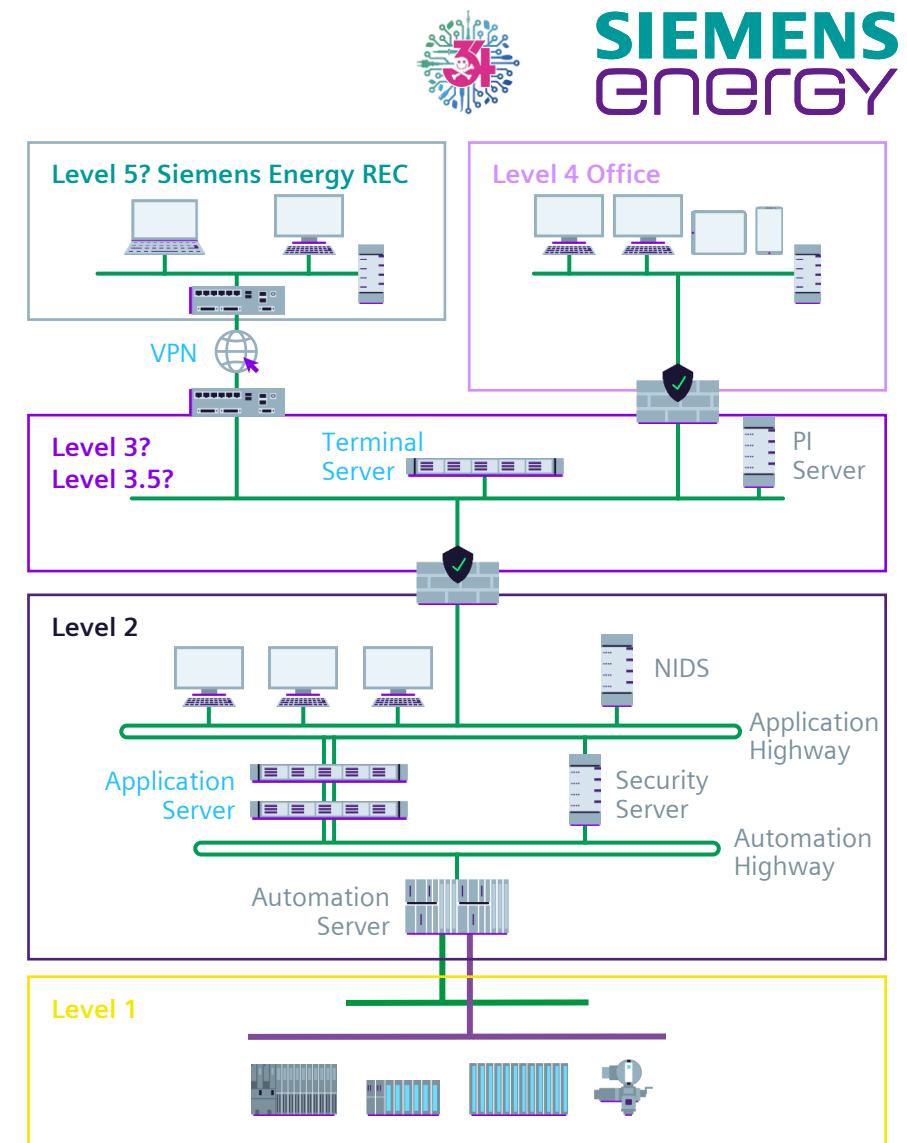
Purdue Model Breakdown

The Purdue Enterprise Reference Architecture is a useful generalized conception of how plants go together.

But it's a reference architecture, not a mandatory reflection of reality.

For example, in T3000, there is a built-in historian and often operators add a PI Server as shown. So what does that make that DMZ with the Terminal Server?

Also, many sites connect to Siemens Energy without sending that connection over their corporate enterprise network first.



How it's supposed to be made secure



SIEMENS
ENERGY

- cRSP Router supposed to be configured to block all external connections except from the Siemens Energy cRSP IPSec VPN.
- T3000 Firewall supposed to only permit RDP.
- Operators are supposed to change the passwords.

Default configuration risks due to drifting security controls over time.



SIEMENS
ENERGY

- Some T3000 updates need SSH from Terminal Server to Application Server and/or Automation Server.
- Some sites use this same remote access path to update 3rd party devices or systems connected to T3000.
- Things changed for service/maintenance are often forgotten.
- No mechanism in T3000 UI to check – requires site personnel to manually check at the device level.
- Passwords are rarely changed. Often the password from commissioning is still in use years or decades later.
- Patches are deployed on long delays – and often only to devices running a full OS, not networking devices which can also have CVE's.
- Monitoring tools rarely checked. Few sites have teams on site who understand cybersecurity monitoring even if they were checking.



Today's exercise.

A realistic scenario

A power plant recently had some trouble with a Simatic S7 PLC being used to control a fuel valve on the fuel skid that supplies natural gas to the gas turbine. The fuel skid vendor does not have a remote access system installed at the plant. Concerned that the CS3000 was faulty, the S7 PLC was connected directly to the Automation Highway for remote troubleshooting.

As the fuel skid vendor did not have their own remote access system, they asked the plant operator to open an SSH connection to the Terminal Server to function as a jump server to remotely connect to the PLC from the Application Server. The plant was shut down during this operation.

Eventually the vendor recovered operation of the PLC via the remote work. As CS3000's were backordered and the plant was losing millions per day of downtime, the vendor reconfigured the PLC to communicate with the Application Server directly to allow the plant to restart.

**But then they
forgot to
turn off web-
facing SSH
on the
Terminal
Server...**

Zoomed In T3000 Network Diagram

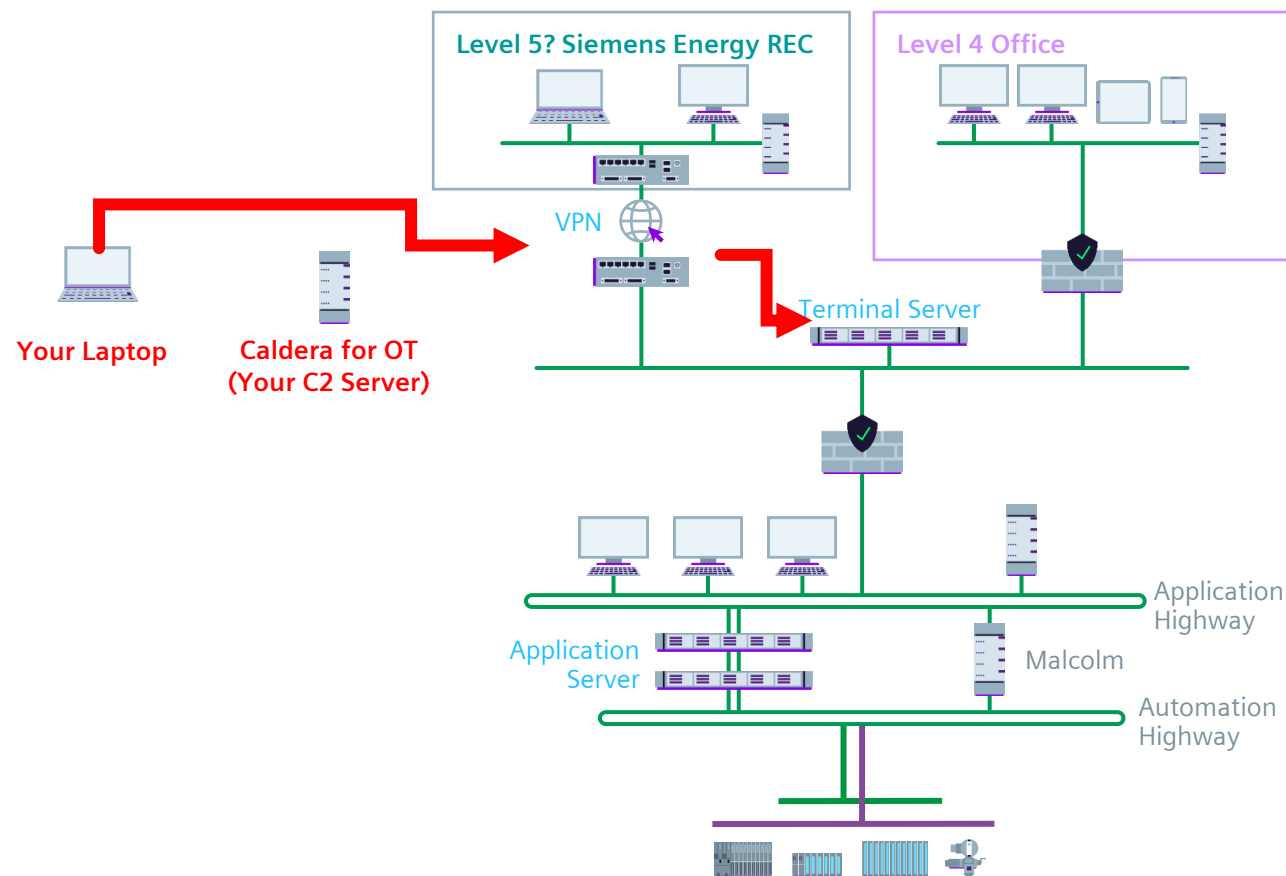
Where are we going in this exercise?



SIEMENS
ENERGY

0. Discovery

Use Censys to find your target Terminal server using publicly-available tools.



Discovery

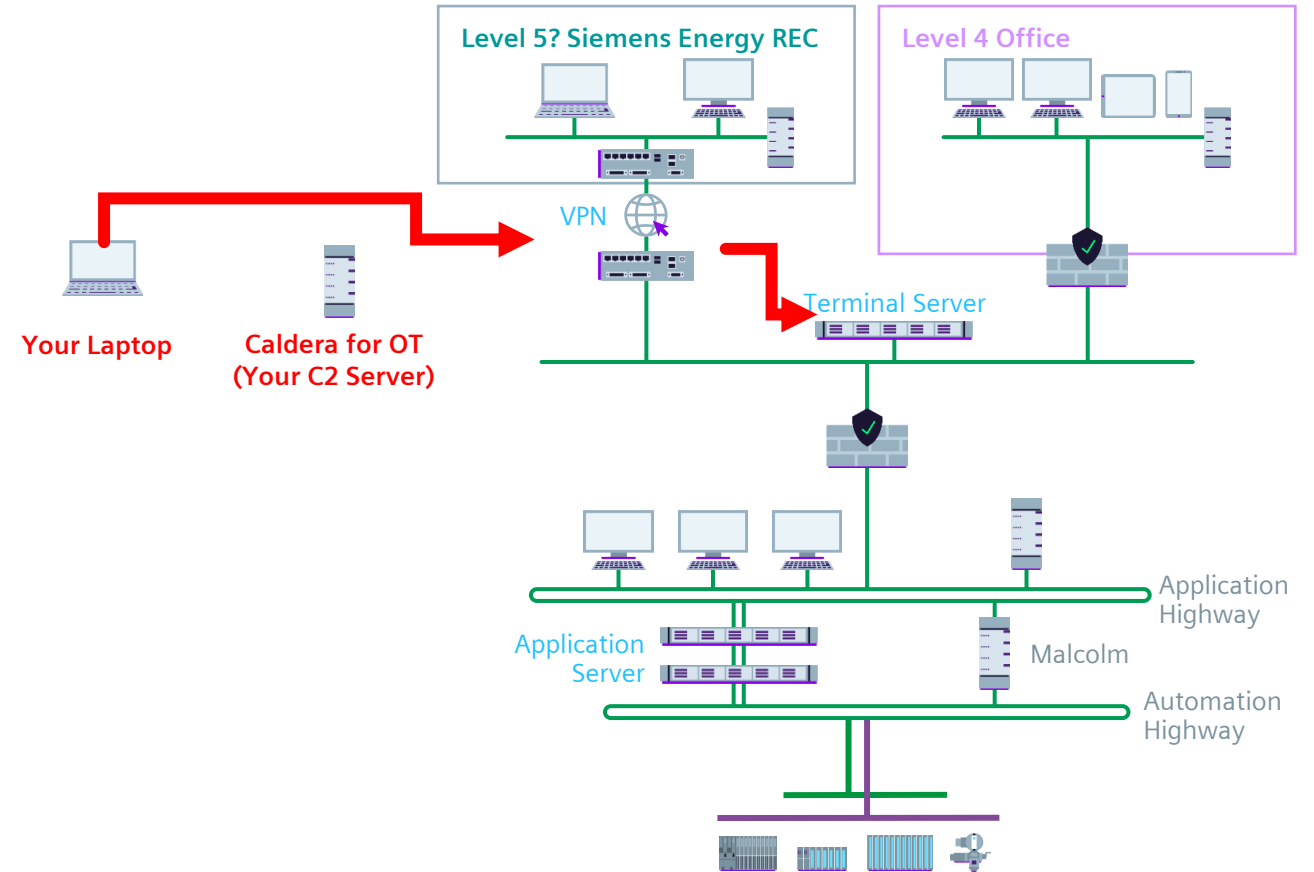
Zoomed In T3000 Network Diagram

Where are we going in this exercise?



1. Initial Access

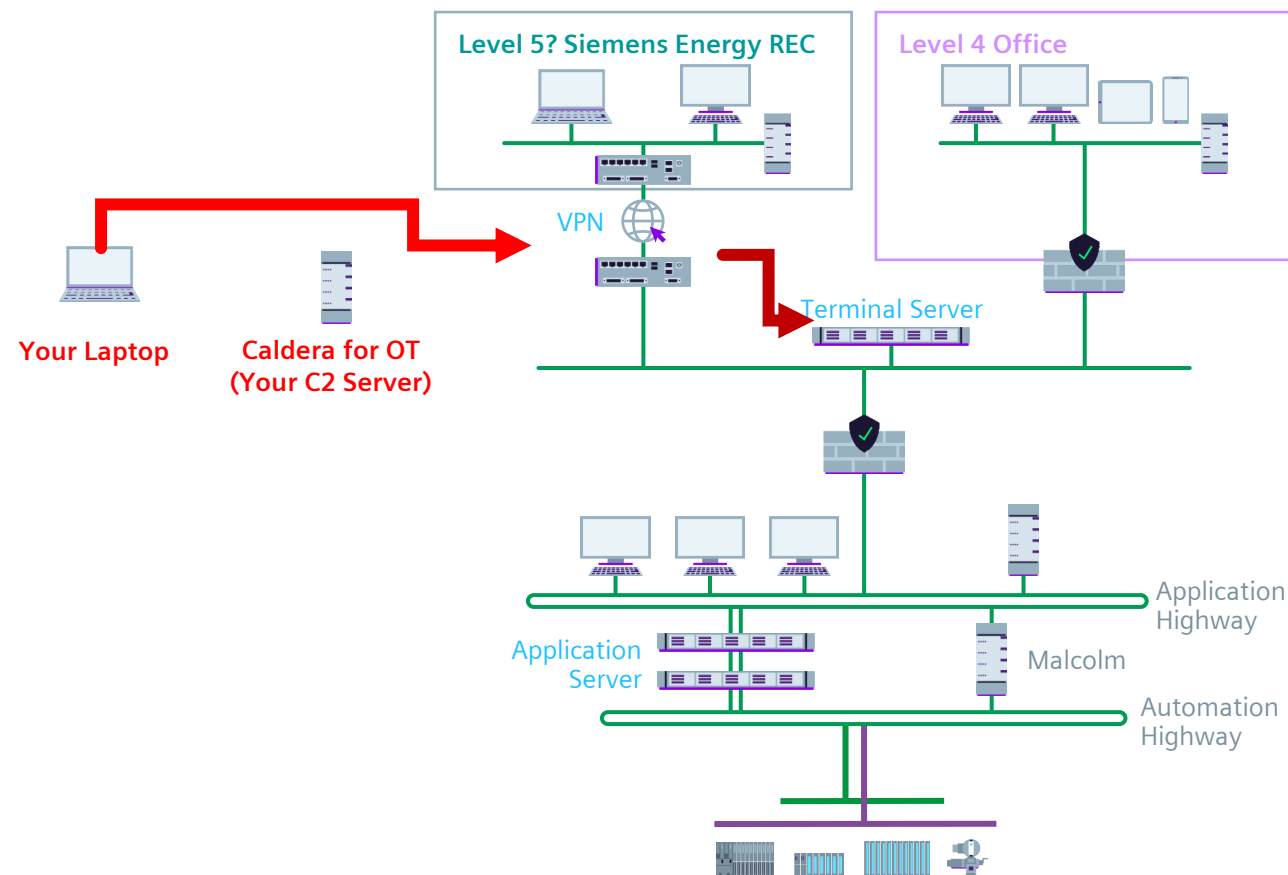
Because the edge router has SSH open and the Terminal Server has no rate-limit on password attempts, we use a standard dictionary attack method using Hydra to get on to the Terminal Server.



Initial Access



We find additional credentials and useful information saved to the Terminal Server by the prior technician team who did not delete the files after they finished their work (a common error).



Collection

Zoomed In T3000 Network Diagram

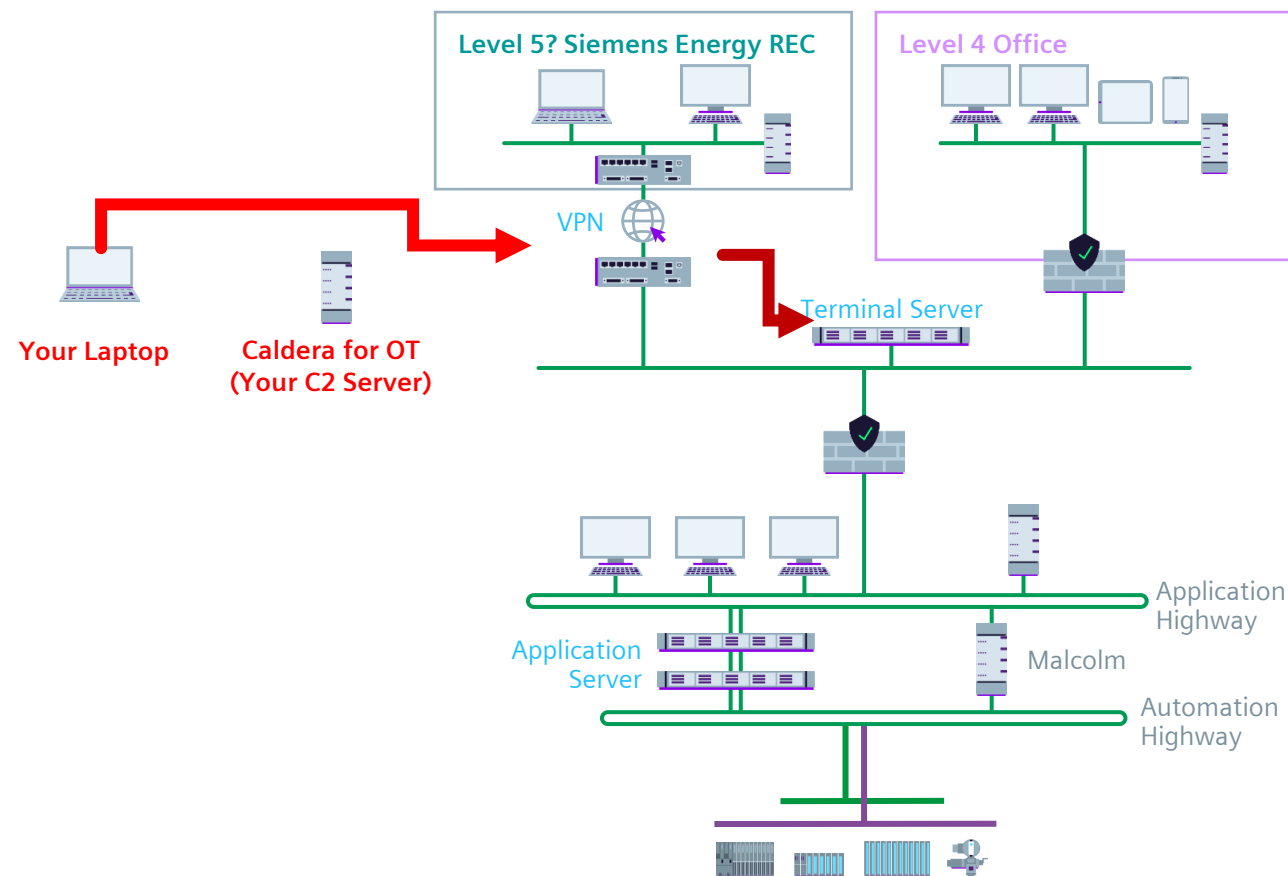
Where are we going in this exercise?



SIEMENS
ENERGY

3. Inhibit Response

We make a small Windows Defender rule update to have Defender not watch our agent's behavior.



Inhibit Response
Function

Zoomed In T3000 Network Diagram

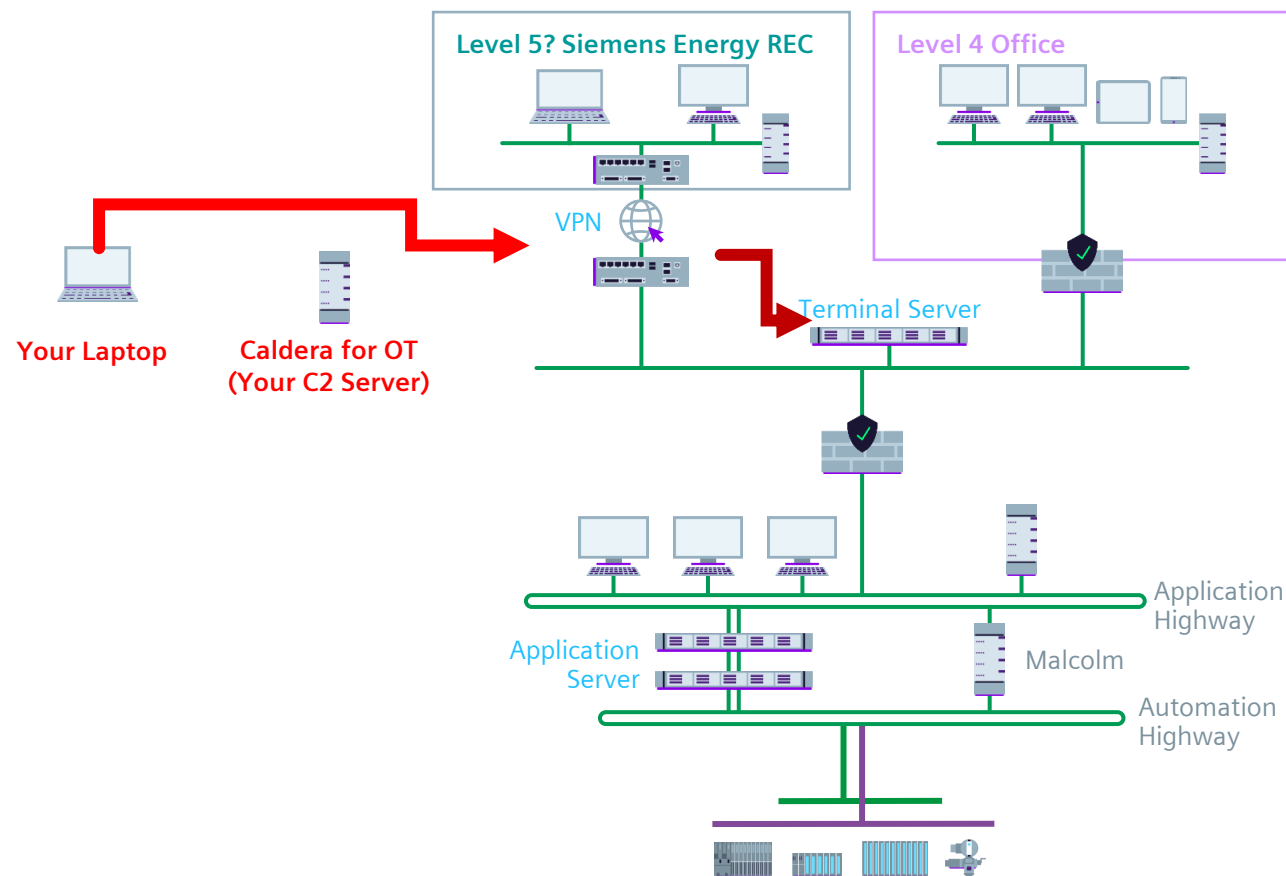
Where are we going in this exercise?



SIEMENS
ENERGY

4. Evasion

We kill the process running on the standard Windows RDP port so we can send our C2 commands over the standard RDP port unnoticed.



Evasion

Zoomed In T3000 Network Diagram

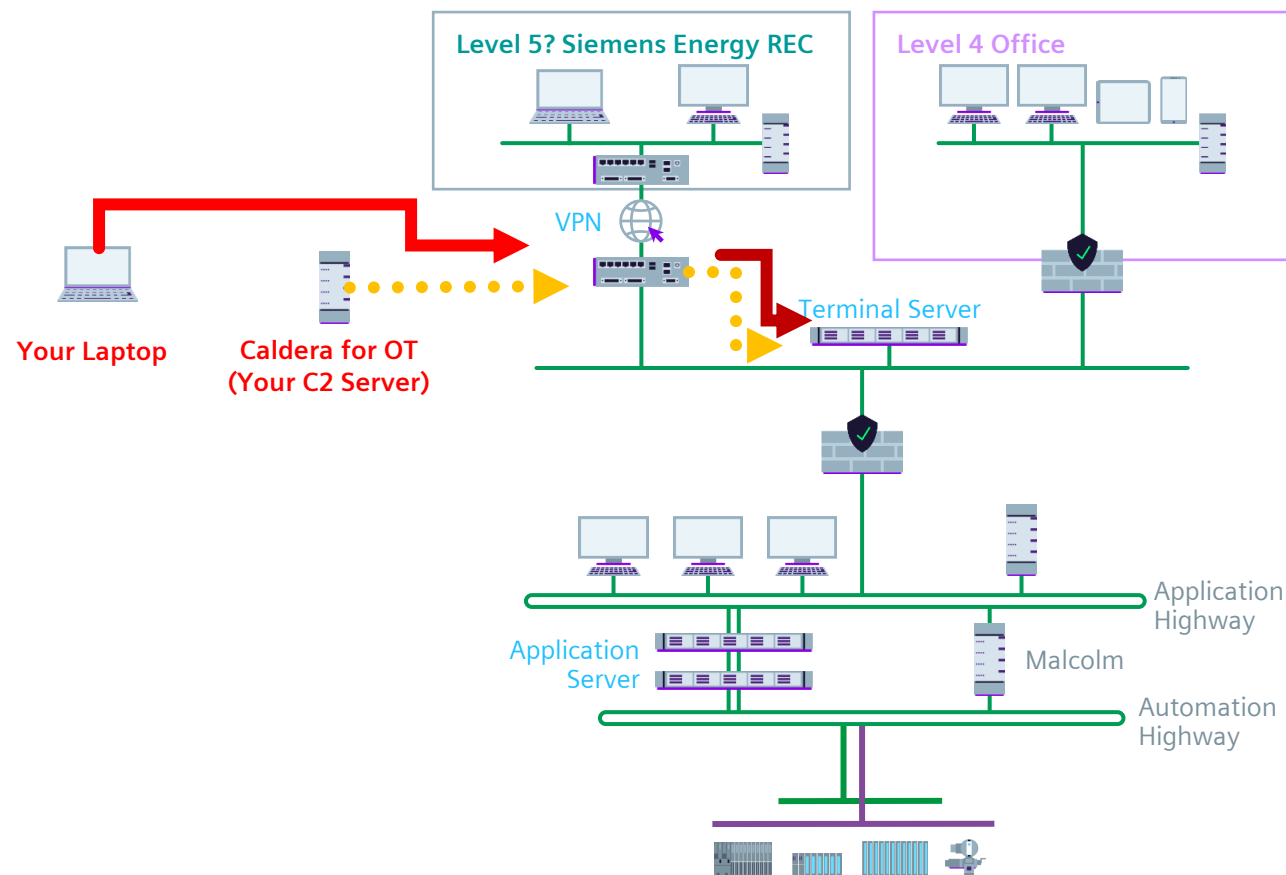
Where are we going in this exercise?



SIEMENS
ENERGY

5. Command and Control and Persistence

We use our C2 server (Caldera for OT) to deploy the pivot agent onto the Terminal Server so that it can relay commands further into the T3000 network.



Command and
Control

Persistence

Zoomed In T3000 Network Diagram

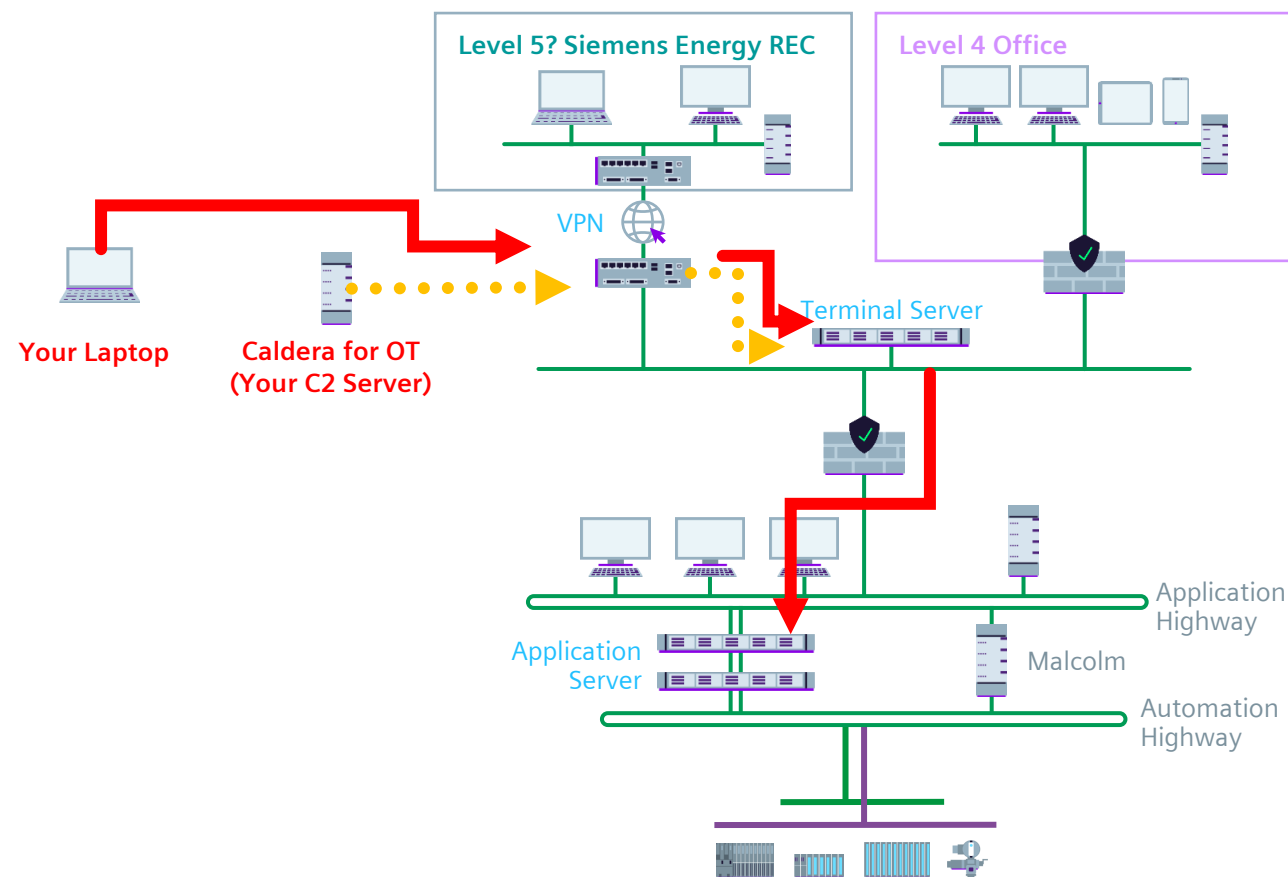
Where are we going in this exercise?



SIEMENS
ENERGY

6. Lateral Movement

We use our presence on the Terminal Server and knowledge about the Application Server's likely open ports to cross the T3000 firewall without detection and gain access to the Application Server.



Lateral
Movement

Zoomed In T3000 Network Diagram

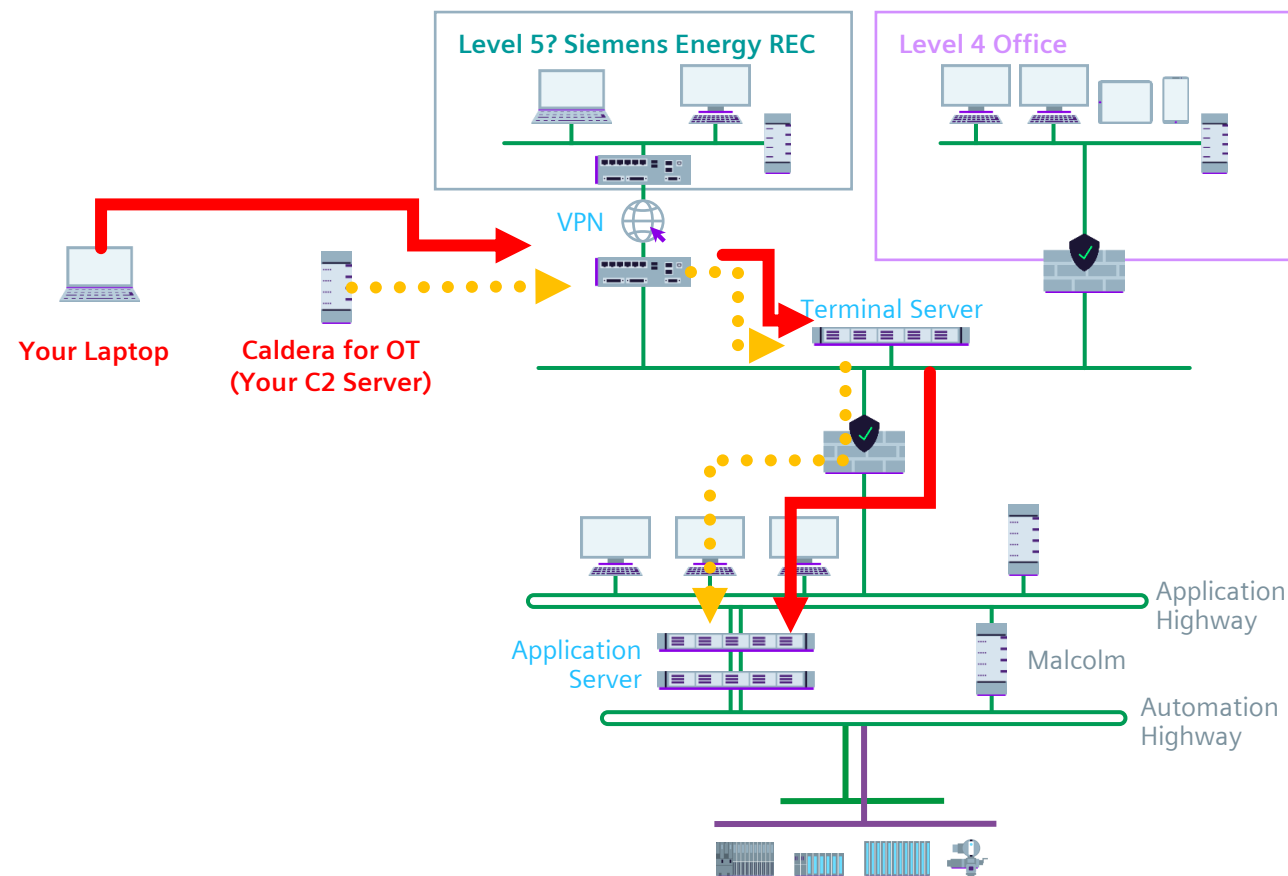
Where are we going in this exercise?



SIEMENS
ENERGY

7. Deploy Agent on App Server

We use our presence on the Terminal Server and knowledge about the Application Server's likely open ports to cross the T3000 firewall without detection and gain access to the Application Server.



Command and
Control

Persistence

Zoomed In T3000 Network Diagram

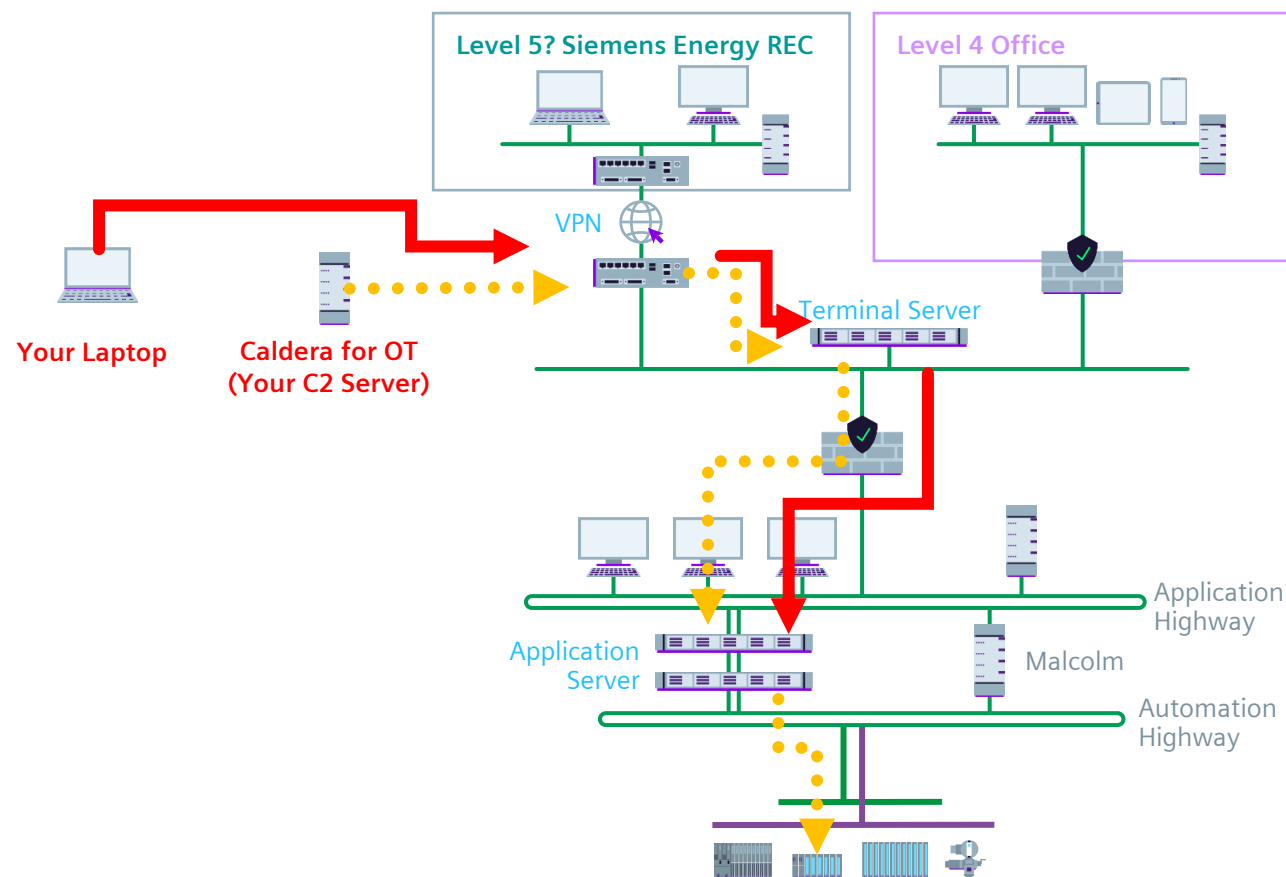
Where are we going in this exercise?



SIEMENS
ENERGY

8. Issue malicious PLC Stop Command and Clean-up to avoid detection

We use our presence on the Terminal Server and knowledge about the Application Server's likely open ports to cross the T3000 firewall without detection and gain access to the Application Server.



Impair Process
Control



SIEMENS
energy

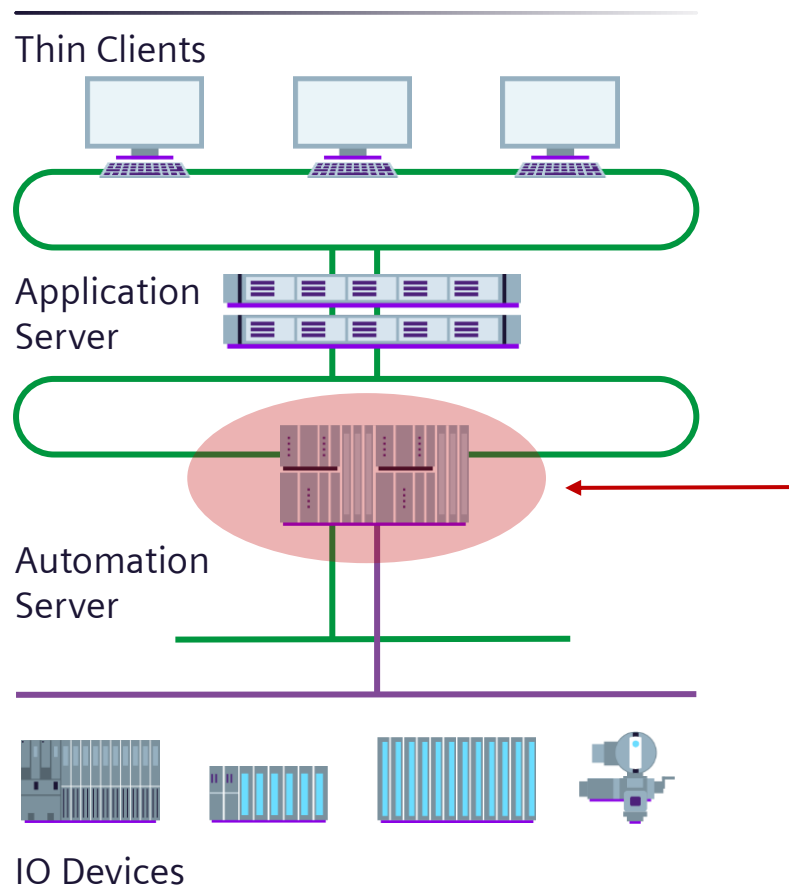
A note from our lawyer...



We're assuming one thing to make sure this exercise does not turn into real hacks...



SIEMENS
ENERGY



We're assuming the Automation Server (AS3000) is not here for purposes of this workshop.

This means that the last step where a PLC stop command is sent is not 100% realistic – you can't do that from the Application Server in real life and configuring the Application Server to communicate directly with the PLC even for a single plant start is not realistic. **We won't show you how to hack the Automation Server.**



Terminal Server Risk is Usually RDP

More Doors Than We Use Today

Especially problematic for Windows Server 2016 and older.

RDP is the more common exposure

Windows Server ships with RDP ready to go — no extra install, no extra step. SSH doesn't: Server 2016 needs Win32-OpenSSH added deliberately.

RDP is always open on a T3000 Terminal Server, not SSH. We used SSH and a dictionary today because it's a reliable, repeatable demo you can safely hand to a big audience.

RDP's documented attack paths are real, but they're memory-corruption bugs, not a login prompt — not something you run live in a classroom.

**SSH is the
easier demo.
RDP is the real
back door.**

RDP's Record, SSH's Record



SIEMENS
ENERGY

RDP: repeat offender

DejaBlue and the December 2024 pair of RDP RCEs means there have been three unauthenticated RCE's against Windows Server 2016's RDP stack in five years.

SSH: no equivalent (yet)

- The one notable Win32-OpenSSH bug needs a valid login and a click first.
- RegreSSHion, 2024's big OpenSSH RCE, doesn't touch Windows at all.

● **RDP has repeatedly skipped the password. SSH hasn't.**

How these risks are supposed to be managed



Common/Default Solutions Disabled in T3000:

- Failed password lockout.
- Dynamic ACL's on switches tied to a NIDS.

T3000 Approved Controls that would be effective against today's exercise:

- Application Whitelisting
- Next Generation Firewalls
- NDR / NIDS / Tenable.ot
- Monitoring on-site SIEM
- Configuration Change Management
- 2FA

A Firewall That Can't Read What It's Passing



SIEMENS
ENERGY

Stateful firewall — the T3000 default

Tracks connection state — source, destination, port, TCP flags. Once a session on port 22 is established, everything inside it is just bytes to the firewall.

NGFW — what most T3000's lack

- Decodes SSH itself: flags anomalous logins, enforces allow-lists of commands or destinations.
- The layer that would notice brute-forcing on port 22.



Most T3000 systems ship with the stateful firewall — the deep-packet-inspection layer is an add-on most sites never buy.

What Layer 7 Awareness Actually Buys You

STATEFUL FIREWALL (T3000 default)



NEXT-GEN FIREWALL (NGFW)



SSH on port 22 is opaque above Layer 4 to a stateful firewall — authentication attempts, command payloads, and file transfers all pass unexamined.



Let's find your targets!

Step 1) Go to <https://github.com/hydrosam/DEFCON34-ICS-Workshop>

Step 2) Download *lps.txt* & *ssh-recon.sh*

Step 3) Find your Terminal Server IP using the envelope found at your stations



SIEMENS
energy

Getting In: A Dictionary Attack on the Terminal Server





Five Steps to a Foothold

Before you start

You need: your assigned VM's IP, dictionary.txt from the Git, and a laptop running Linux or Windows.

- 1 The Target & the Dictionary
- 2 Why This Works Here
- 3 Hydra on Linux
- 4 Hydra on Windows (WSL)
- 5 The Real-World Math



The Target and the Dictionary

Know Your Target Before You Attack



SIEMENS
energy

Your target

- A Windows Server 2022 Terminal Server.
- Username: Administrator.

Your toolkit

- Linux users: Hydra, already installed on Kali or installable on other distros.
- Windows users: WSL + Ubuntu.
- One dictionary file — dictionary.txt.



You'll notice the dictionary.txt file is small, in real life this would be much larger.

Setup for Windows

PowerShell



```
# Install / activate WSL
```

```
PS C:\> wsl --install
```

```
# Reboot then set WSL user name and password
```

```
PS C:\> wsl
```

```
hannes@hannes-pc $
```

Attackers Are Not Guessing Blind



SIEMENS
ENERGY

- Real attackers rarely guess randomly — they use lists built from breach dumps, defaults, and site-specific patterns.
- Today's dictionary has exactly 180 entries: one real password per student plus some filler.
- Hydra tries them in order until one works.



SIEMENS
energy

Why This Works Here

No Lockout.



SIEMENS
energy

Windows Server out of the box

Server 2022 ships with account lockout available — 10 bad attempts trips it.

T3000 Terminal Server as deployed

- Lockout is turned off. A locked-out operator during an incident is worse than a slow brute-force attempt — so availability wins.

● **Unlimited guesses isn't an oversight — it's a trade-off.**



Hydra on Windows (via WSL)

Same hydra Command

engineer@DESKTOP:~\$ (Ubuntu on WSL)

```
PS C:\> wsl
```

```
hannes@hannes-pc $ sudo apt update && sudo apt install -y hydra
```

```
# your dictionary.txt is under /mnt/c/...
```

```
$ cp /mnt/c/Users/student/Desktop/dictionary.txt .
```

```
$ hydra -l Administrator -P dictionary.txt ssh://10.60.4.21 -t 4 -f -o found.txt
```

```
[22][ssh] host: 10.60.4.21  login: Administrator  password: XXXXXXXXXX
```

Confirm the Foothold

student@DESKTOP:~\$ (Ubuntu on WSL)

```
$ ssh Administrator@10.60.4.21
```

```
Administrator@10.60.4.21's password: *****
```

```
Last login: Wed Jul 29 2026 from 10.60.4.9
```

```
$ whoami
```

```
termsrv-01\Administrator
```



SIEMENS
energy

The Real-World Math



180

maximum attempts needed — one per dictionary entry.
Hydra burns through all of them in well under a minute.

~432,000 yrs

worst case to brute-force ONE 8-character
upper/lower/digit password with no
dictionary, at a generous 16 guesses/sec

2.18×10^{14}

possible 8-character combinations (62^8) an
attacker without your dictionary has to
search

If you had an RCE but no good dictionary, you'd have another path in...



Command Prompt — Administrator

```
# any RCE that can land a SYSTEM shell -- no valid password used
C:\> reg save HKLM\SAM sam.save
C:\> reg save HKLM\SYSTEM system.save
```

kali — remote

```
# or remotely, once you hold admin-equivalent access:
$ secretsdump.py -sam sam.save -system system.save LOCAL
[*] Target system bootKey: 0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d
[*] Dumping local SAM hashes
Administrator:1001:aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117ad06bdd830b7586c:::

# cracked OFFLINE from here -- no lockout policy, no rate limit, no dictionary needed
```



0

passwords required if you have an RCE

79,000

internet-facing Remote Desktop Licensing
hosts Censys found still exposed to CVE-
2024-38077

Jan 12, 2027

when Windows Server 2016 stops
receiving patches for any of this — for
good



SIEMENS
energy

MITRE Caldera for OT: Automated Adversary Emulation for ICS





Six Concepts Before the Live Exercise

- 1 What CALDERA Is
- 2 Access & Agents
- 3 Abilities & Adversaries
- 4 Operations & Agent Tuning
- 5 What Upstream C4OT Supports
- 6 Extending It With S7comm Plugin



What is CALDERA

Automated Adversary Emulation



Built on MITRE ATT&CK

Caldera automates actions that a red teamer would normally execute manually. Instead of running every command one at a time, we package those actions into repeatable attack workflows.

Every operation uses three main pieces:

- **Agents** — programs running on the target systems
- **Abilities** — individual commands or attacker actions
- **Adversary profiles** — ordered attack playbooks

Choose the agents, select an adversary profile, and start the operation. Caldera executes the chain, collects information, and records what worked.



Access & Agents

How Operators Access Caldera

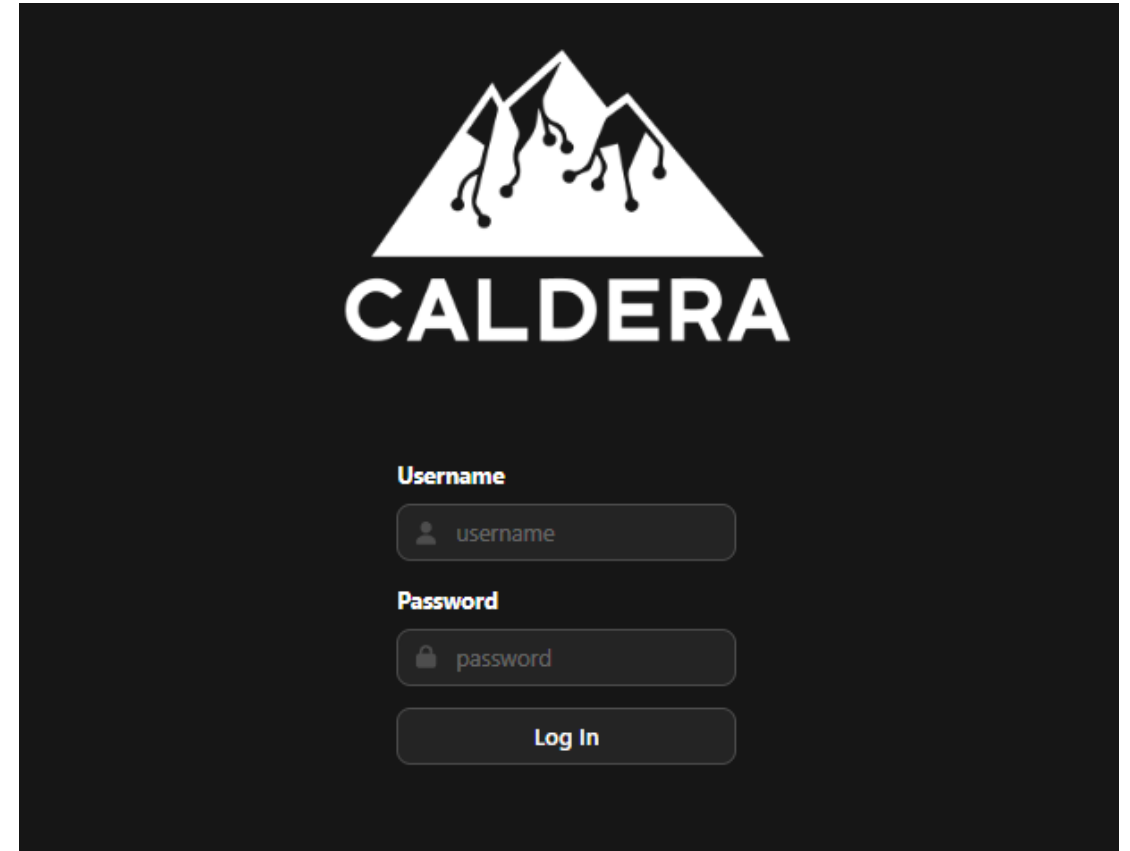


SIEMENS
ENERGY

Caldera runs as a server application with a browser-based operator interface.

After authenticating, the operator uses this interface to configure and control the environment.

The browser is the control interface—not the attack itself.



The Agent Connects Caldera to the Target



The agent is Caldera's connection to the target system.

The agent is a small program running on the compromised machine. It:

- Checks in with the Caldera server
- Receives abilities to execute
- Runs commands using the target
- Sends command output and discovered facts back to Caldera

For this workshop, we use Sandcat, Caldera's default Go-based agent.

Once the agent is connected, Caldera can task that machine as part of an operation.

Important distinction

The agent is not the attack itself.

It is the delivery and execution mechanism that allows Caldera to run the abilities defined in an adversary profile.



Abilities & Adversaries

Abilities: Caldera's Reusable Actions

What is an ability?

An ability is one reusable action that Caldera can send to an agent.

It defines:

- What action to perform
- What command or tool to run
- Which platform can run it
- What information it needs
- How to interpret the results

Abilities can use facts already collected during the operation and return new facts for later abilities.

Bottom takeaway

Abilities are the individual building blocks Caldera chains into a complete operation.

Example

Known PLC IP



Run "Read CPU State" ability



Agent executes the S7 tool



CPU state returned to
Caldera

Adversary Profiles: Turning Abilities into a Plan

What is an adversary profile?

An adversary profile is an ordered sequence of abilities that Caldera can run as one operation.

Abilities define an individual actions.

The adversary profile defines which actions run and in what order.

This turns a collection of isolated commands into a repeatable attack workflow.

Caldera calls this sequence the atomic ordering.

ABILITY

= one action

ADVERSARY PROFILE

= the attack plan



Operations & Agent Tuning

Operations: Where Everything Comes Together

An operation is the live execution of an adversary profile against a group of agents.

It brings together:

- **Agents** — where the actions will run
- **Adversary profile** — which abilities Caldera should attempt
- **Fact source** — what Caldera already knows at the start
- **Planner** — how Caldera decides what can run next

HOW THE OPERATION MOVES

As abilities execute, Caldera collects results and creates new facts.

Those facts can unlock later abilities and keep the operation moving forward.

The adversary profile is the plan.

The operation is Caldera carrying it out.

Beacon Timing: Faster Isn't Always Better



Agent Configuration

Beacon Timers (sec) min max

Watchdog Timer (sec)

Untrusted Timer (sec) *

Implant Name

Bootstrap Abilities ⓘ Avoid logs × +

Deadman Abilities ⓘ +

* A global setting that will effect any new or existing agents.

Close Save

Agents check in with Caldera at a configurable interval.

SHORTER INTERVAL

- Abilities execute sooner
- Easier to follow during the workshop
- Creates more frequent C2 traffic
- May produce a more obvious network pattern

LONGER INTERVAL

- Operations take more time
- Creates fewer check-ins
- Can spread activity across hours or days
- Reduces sudden bursts of observable activity

Attackers balance speed against exposure.

Move quickly before access is lost, or slow down to avoid drawing attention?



What Upstream C40T Supports

Caldera for OT: Same Platform, New Protocols

Released September 2023

Standard Caldera is primarily built around enterprise IT, Windows and Linux agents, command execution, and techniques mapped to Enterprise ATT&CK.

Caldera for OT, or C4OT, extends that same platform into industrial environments.

It adds:

- OT-specific abilities and payloads
- Support for industrial protocols
- Techniques mapped to ATT&CK for ICS
- Research developed by MITRE and CISA

The core structure does not change: agents execute abilities, and adversary profiles organize them into an attack chain.

C4OT simply gives Caldera the tools to interact with industrial systems.





6

OT protocols supported today — Modbus, BACnet, DNP3, IEC 61850, and Profinet, with GEMS (a space protocol) in active development

3

protocols in the original September 2023 release — Modbus, BACnet, and DNP3

S7comm

needed to accurately emulate an attack on T3000 and other S7-based ICS's



Extending It With S7comm Plugin

Adding S7comm Support to Caldera

Adapting Caldera's OT plugin model for Siemens S7 workflows

MITRE's Modbus plugin gave us a useful reference for how protocol-specific capabilities can fit into Caldera.

We followed that general structure and added the pieces needed for Siemens S7 communication:

- **Abilities** — Define the S7 actions available inside Caldera.
- **S7 Tool** — Communicates with the PLC over TCP/102.
- **Parsers and Facts** — Return useful results so later actions can build on them.

Bottom line

This extends Caldera's existing framework with Siemens S7-specific capabilities.

Example workflow

01 **Discover PLC**

02 **Identify CPU**

03 **Read Data**

04 **Perform Control Action**



SIEMENS
energy

Let's Do it Together





SIEMENS
energy

Finding the Adversary: Suricata Rules & OpenSearch Hunting in Malcolm





Five Steps to Catching Them

Before you start

You need: a laptop on the lab network, a browser open to your Malcolm instance's OpenSearch Dashboards, and a text editor for writing the rule.

- 1 Write the Suricata Rule
- 2 Load It Into Malcolm
- 3 Watch It Fire
- 4 Hunt After the Fact
- 5 The S7comm STOP Walkthrough



Write the Rule — Before the Attack

Suricata: A Network's Smoke Detector

Open source, maintained by Open Information Security Foundation (OISF)

Suricata is a high-performance network intrusion detection, intrusion prevention, and network security monitoring engine, it watches traffic and tells you when something matches a rule you care about.

It's developed by the Open Information Security Foundation (OISF), a non-profit that got its start with DHS (Department of Homeland Security) seed funding in 2010, and it's released under the GPLv2 (General Public License v2 – 1991), free for anyone to run.

Point it at an interface, load it with rules, and it reads every packet crossing that interface in real time, comparing it against thousands of known bad patterns.



The Cookbook Recipe: Action + header + options



Every Suricata rule (also called a signature) follows the same shape:

1. an action (usually alert, sometimes drop if you're running in IPS mode),
2. a header describing the traffic to match (protocol, source, destination, ports), and
3. options in parentheses describing what to look for and how to label it.

```
alert ip any any -> any any (msg:"GPL ATTACK_RESPONSE id check  
returned root"; content:"uid=0|28|root|29|"; classtype:bad-unknown;  
sid:2100498; rev:7;)
```

- msg is the human-readable alert text,
- content is the pattern to match,
- Sid (signature ID) is the rule's unique ID

Rules Only Help If They're Already Loaded

Suricata doesn't store anything

Detection is not retroactive and alerts need somewhere to go: Malcolm

Suricata inspects traffic as it crosses the wire in real time. Suricata's eve.json file is a firehose, every alert, every flow, every DNS lookup, written to a log file that grows forever and isn't searchable on its own.

A rule written after an attack has already happened will never see that attack, it can only catch the next one. We have preloaded some Suricata rules that are similar to rules in a real T3000 NIDS.

Malcolm ingests Suricata's alerts directly, alongside Zeek's protocol logs and full packet captures, normalizes all three into OpenSearch, and gives you dashboards to search, filter, and hunt through everything Suricata just told you.

**Suricata sees it
and alerts.
Malcolm is
where *you* look.**

Anatomy of a Suricata Rule - Catching the S7comm PLC STOP



① ACTION ② HEADER ③ OPTIONS (everything in parentheses)

```
alert tcp any any -> any 102 (msg:"ICS Alert: Siemens S7 PLC STOP Command Issued"; flow:established,to_server; content:"|03 00|"; offset:0; depth:2; content:"|02 f0|"; distance:2; within:2; content:"|32 01|"; distance:1; within:2; fast_pattern; content:"|00 00|"; distance:0; within:2; content:"|29|"; offset:17; depth:1; dsize:>17; classtype:policy-violation; priority:1; reference:url,attack.mitre.org/techniques/T0816/; metadata:mitre_technique_id T0816, mitre_technique_name Device_Restart_Shutdown, deployment ICS; sid:9000001; rev:2;)
```

① · ACTION

alert

What Suricata does on a match. alert logs it and moves on; in inline/IPS mode you'd use drop to block. Here we only want to be told, not to break live plant traffic.

② · HEADER

tcp any any -> any 102

Which packets to inspect. Protocol tcp, from any source IP/port, -> to any host on port 102, the ISO-on-TCP port S7comm rides on. The arrow fixes direction: traffic heading into the PLC.

③ · OPTIONS

(...)

What to match inside the packet and how to label the hit. content byte-matches walk the packet header-by-header to function code [29] at offset 17; dsize, flow and fast_pattern tune accuracy and speed; classtype / priority / sid / metadata label it and tie it to MITRE ATT&CK for ICS.

WHAT THIS RULE IS

A **signature**: a byte fingerprint of one exact message on the wire. It walks the S7comm stack `TPKT |03 00| → COTP |02 f0| → S7 Job |32 01|` and checks the function-code byte that follows.

WHAT IT DETECTS

An S7comm Job carrying function code **0x29**, the command that halts a running CPU. Operators stop a live PLC on purpose, so a match is a high-fidelity **"someone told the PLC to STOP."** Fires priority 1, tagged **MITRE T0816** (Device Restart/Shutdown), the exact red-team action this lab hunts.

Suricata Is The Engine

analyst@sensor:~\$



```
# install Suricata and point it at your monitored interface
sudo apt install suricata jq
sudo suricata-update
20 rule files processed. 1 rule files skipped.
Rules successfully loaded. Written to /var/lib/suricata/rules/suricata.rules.

# start the engine
sudo systemctl restart suricata

# watch alerts as they happen
sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
```



Malcolm: Making Sense of the Traffic

Purple Team Lab Guide | DEF CON 34 Workshop





Five Steps to Your First Hunt

Before you start

Open the Remote_Access_Quick_Reference.txt file you found on the Terminal Server.

In the file you can find the IP, username and password for your Malcolm instance.

C:\Users\Public\Documents\Remote_Access_Quick_Reference.txt

1

What Is Malcolm?

2

Where Suricata Fits In

3

Logging In & Watching Live Traffic

4

Engines & Interfaces

5

Tuning What It Watches



What Is Malcolm?

One Product Connecting Many Open-Source Tools

Built by CISA and Idaho National Laboratory

Malcolm is a network traffic analysis tool suite — it doesn't generate its own detections so much as it takes the open source tools that already do (**Zeek**, **Suricata**, **Arkime**, **OpenSearch**) and wires them together into one workflow.

Feed it **full packet captures**, **Zeek logs**, or **Suricata alerts** — by upload or live capture — and Malcolm normalizes, enriches, and indexes everything automatically, then hands it to you through two interfaces: **OpenSearch Dashboards** for visual, ad-hoc exploration, and **Arkime** for finding and replaying the exact sessions behind a suspected incident.

Apache 2.0 Licensed!

The logo for Malcolm, featuring the word "Malcolm" in a white, bold, sans-serif font with a black outline. The letter "o" is replaced by a circular icon containing a stylized, golden, interlocking knot or infinity symbol.

How is Malcolm Hooked Up?

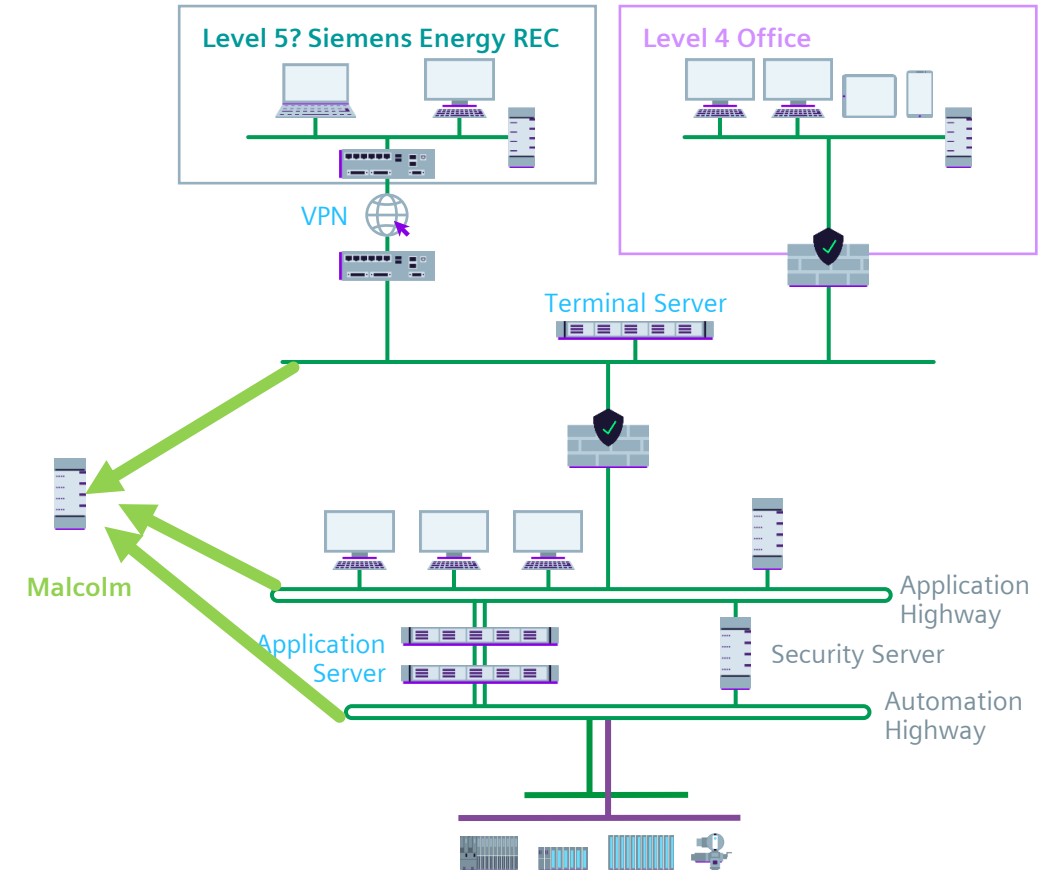


SIEMENS
ENERGY

Traffic Mirroring

We mirror the traffic from the Terminal Highway, Application Highway, and Automation Highway to Malcolm.

This mirroring is accomplished with the Scalance LAN-Port Mirroring feature. Similar in functionality to a SPAN port.





SIEMENS
energy

Where Suricata Fits In

Suricata alerts are one of three inputs to Malcolm



Suricata alerts are one of three inputs

Malcolm accepts exactly three kinds of input:

1. full packet captures,
2. Zeek's protocol logs, and
3. Suricata's alerts.

Filebeat and Logstash pick all three up, normalize and enrich them, and index everything into OpenSearch.

From there, every Suricata alert is a searchable, dashboardable event inside Malcolm, correlated against the Zeek session and raw packets that surrounded it, instead of sitting alone in a log file on the sensor.



**We pre-configured your
Malcolm instances to
watch while we did the red
team exercise.**

Let's see what it found!



Logging In To Malcolm

First, Log In



Malcolm Credentials: C:\Users\Public\Documents\Remote_Access_Quick_Reference.txt

52.36.11.179

This site is asking you to sign in.

Username

Password

Sign in

Cancel

Navigation

General Network Logs

Overview

Total Number of Logs

154

zeek - Logs

Log Source

Log Source	Last Ingested	Count
malcolm	Aug 3, 2026 @ 00:34:15.664	154

Rows per page: 5

Total Log Count Over Time

zeek

Log Type

Data Source	Log Type	Count
zeek	conn	49
zeek	notice	34
zeek	known_hosts	23
zeek	ssl	22
zeek	known_services	19
zeek	http	3
zeek	files	1
zeek	ocsp	1
zeek	redis	1
zeek	software	1

Rows per page: 10

Application Protocol

Application Protocol	Protocol Version	Count
ssl	-	35
tls	TLSv13	22
http	-	7
http	1.1	3
redis	-	2
X.509	-	1

Rows per page: 10

Actions and Results

Protocol	Action	Result	Count
tls	Connect	Success	21
tls	Resume	Success	1
http	GET	Not Modified	2
http	GET	Success	1
redis	zremrangebyscore	Success	1

Rows per page: 10

HTTP / MQTT

MySQL • NTLM • NTP • OSPF • PostgreSQL • QUIC • RADIUS • Redis • RDP • RFB • SIP • SMB • SMTP • SNMP • SSH • SSL / X.509 Certificates • STUN • Syslog • TDS / TDS RPC / TDS SQL • Telnet / rlogin / rsh • Tunnels

ICS/IoT Protocols

BACnet • BSAP • ANSI C12.22 • DNP3 • EtherCAT • EtherNet/IP • GENISYS • GE SRTP • HART-IP • IEC 104 • Modbus • Omron FINS • OPCUA Binary • PROFINET • ROC Plus • S7comm

Three Different Views

Zeek, Suricata, and Arkime each see something different

Zeek reads every session and writes structured logs describing what it saw i.e. connections, DNS lookups, file transfers.

Suricata reads the same traffic looking for matches against its rules and raises alerts when something fires.

Arkime captures and indexes the sessions themselves, so you can go back and replay the actual packets behind any of it.

Malcolm runs all three against the same traffic and merges what they find.





After the Fact — Hunting in OpenSearch

When There's No Rule Waiting

Hunting is retroactive

Suricata only flags what it was told to look for. Everything else — traffic nobody wrote a rule for — is still sitting in Malcolm's OpenSearch indices, waiting to be found.

Hunting means starting from a hypothesis (“something touched my PLCs”) and querying your way to proof, instead of waiting for a red banner. It's slower than an alert, but it's the only option once an attacker uses a technique nobody anticipated.

**If Suricata
didn't catch it,
OpenSearch
still saw it.**

The Scenario, the Protocol Fact



SIEMENS
energy

What happened

Something on your network sent an S7comm STOP command to a PLC today.

No alert fired — it isn't in the ruleset. You have to find it.

What you're looking for

- S7comm function code 0x29 (decimal 41) is “PLC Stop” — the command that halts a running CPU.
- Engineers stop a live CPU deliberately, not by accident.

● **Function code 41.**



SIEMENS
energy

Hunting

Another Team, Another Trail

No rule for this one either

The rest of today's exercises assumed a network-visible attack: a brute-force flood, a PLC stop command.

Some techniques don't work that way — they live on the host.

- A renamed process.
- A security tool told to look away.
- A familiar service quietly moved off its usual port.

Suricata and Zeek can't read a process name off disk. But every one of these actions still writes something down: an event log entry, a changed listener, a protocol that doesn't match the port it's running on.

**Masks fool
people. They
don't fool
packets.**

The Signal You Don't Need Host Logs For

Zeek reads traffic, not port numbers

Zeek's Dynamic Protocol Detection identifies a protocol from its actual bytes on the wire, not from the port carrying it.

If something takes over a well-known port without speaking that port's real protocol, Zeek's own log for that protocol simply won't contain a matching entry for the session — it gets tagged with whatever protocol it actually is, or nothing at all.

**Zeek reads
the wire, not
the label.**

Step 1: Scope to the Port

OpenSearch Dashboards – Discover

```
# OpenSearch Dashboards -> Discover -> zeek-* index pattern
```

```
destination.port:3389
```

```
every session that claims to use the RDP port -- real RDP clients included
```


Step 2: Check What Zeek Saw

OpenSearch Dashboards – Discover

```
# legitimate RDP sessions log as event.dataset:rdp -- exclude those
```

```
destination.port:3389 AND NOT event.dataset:"rdp"
```

1 result

```
source.ip:      10.60.4.40    (Application Server)
destination.ip: 10.60.4.21    (Terminal Server)
event.dataset:  http         <- Zeek's actual read on the traffic
```

```
# confirm the real event.dataset value in your own sidebar -- it won't
# always say the same thing
```

The Masquerade Hunt, in One Place



cheat sheet

```
# today's queries, in order
```

```
destination.port:3389
```

```
-> everything claiming to be the RDP port
```

```
destination.port:3389 AND NOT event.dataset:"rdp"
```

```
-> port 3389 traffic Zeek did NOT confirm as real RDP
```

```
# a port is a number a process picks. event.dataset is what Zeek  
# actually saw on the wire.
```



What could have prevented this attack?

What did we see?



SIEMENS
ENERGY

- It all starts with one firewall rule misconfigured.
- A single misconfiguration of a stock T3000 leaves the rest of your security posture up to chance and time.
- It is possible to send a malicious PLC command in a way that looks benign.
- RCE's just make things faster, they don't materially contribute to this attack path.

What would stop this attack?



SIEMENS
ENERGY

- 2FA on T3000 user accounts
- NGFW
- Application Whitelisting
- Strong, random, protected credentials
- Routing cRSP over corporate IT rather than directly to the web.

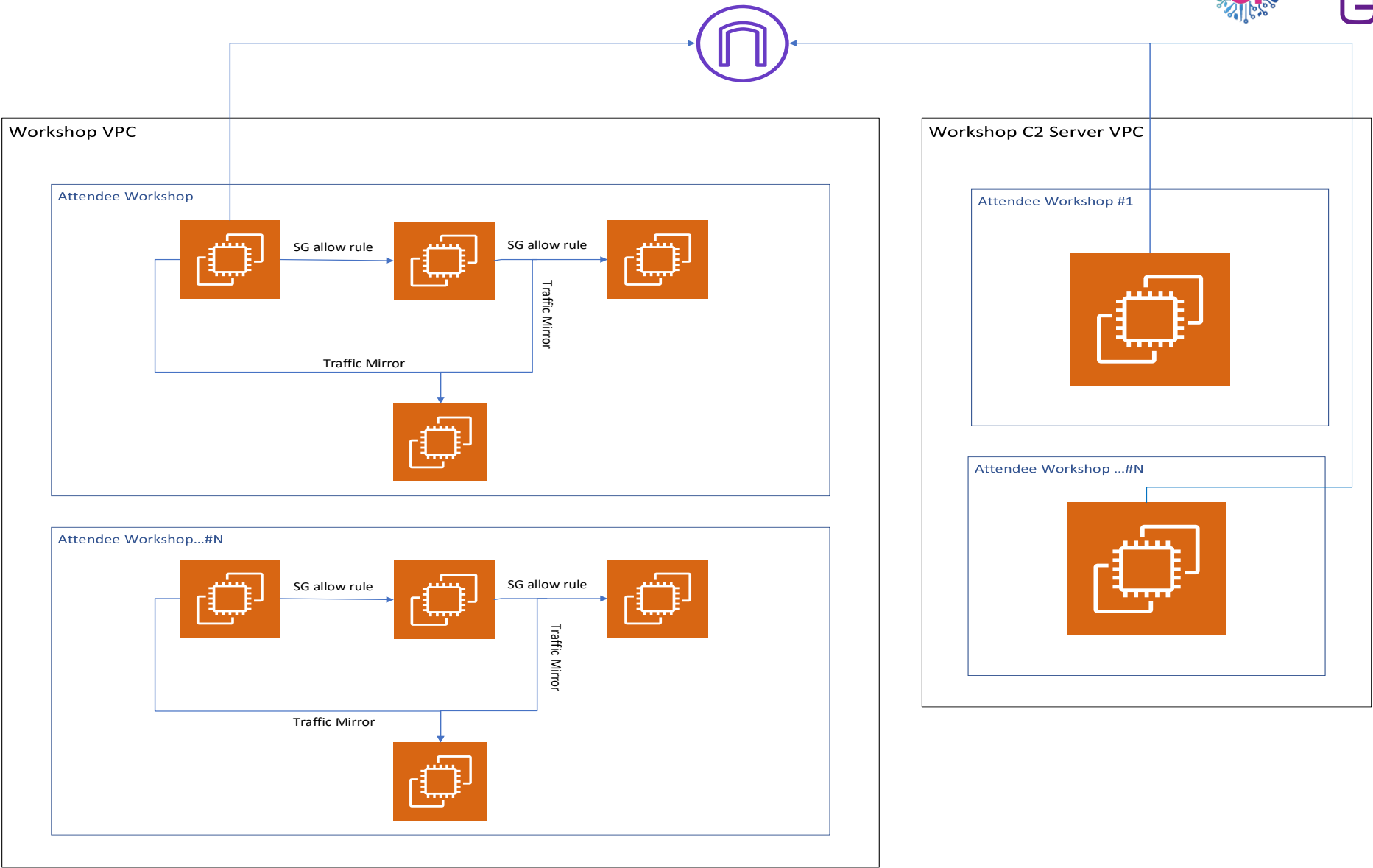


How did we put this together?

Workshop Cloud Architecture – Visualized



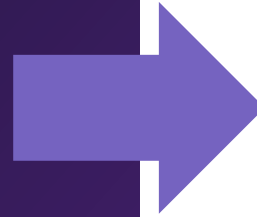
SIEMENS
ENERGY



How can you put this together?

Infrastructure

- **An AWS account and ~\$35**
- **2 VPCs**, an OT network and your C2 network
- **4 subnets**,
 - 10.99.10.0/24 -> OT public subnet
 - 10.99.11.0/24 -> C2 public subnet
 - 10.99.12.0/24 -> App Highway
 - 10.99.13.0/24 -> Terminal Highway
- **5 EC2 instances**, term server, app server, c2 server, emulated PLC, and Malcolm instance
 - Terminal Server -> t3.small or equivalent
 - App Server -> t3.small or equivalent
 - C2 Server, PLC -> t3.small or equivalent
 - Malcolm Instance -> m7i.2xlarge



Networking

- **C2 server** will live on the **C2 network VPC**
- All other machines will live in the **OT network VPC**
- **Traffic mirroring sessions** with Malcolm as a target for all ENIs that you'd want to visualize

Software

- **Malcolm**
- **MITRE Caldera** with OT modules
- Python-snap7 python library



SIEMENS
ENERGY

We'd love your feedback:

Sam Miorelli – sam.miorelli@siemens-energy.com, @sammiorelli on X

Pedro Cabrera – pedro.cabrera@siemens-energy.com

Hannes Heck – hannes.heck20@gmail.com

Hannah Krebs – hannah.krebs@siemens-energy.com

Matt Parrott – matt.parrott@siemens-energy.com

Jordan Sanchez – jordans2604@gmail.com

Anthony Sevarino – anthony.sevarino@siemens-energy.com